

Construction d'une API avec Spring Boot



Présentation



Spring Boot est un Framework Java permettant de développer des API backend. Il permet en outre:

- La manipulation des données – lecture et écriture
- La mise en place d'un système de routage simple

Environnement de développement



Pour développer nos API sur Spring Boot, nous utiliserons Eclipse.

Spring Boot possède un serveur HTTP Tomcat intégré.

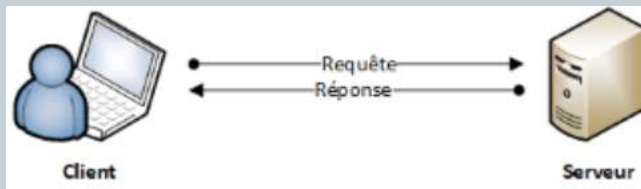
Le programme s'exécute comme une application Java.

Rappels sur le protocole HTTP



Quelques éléments sur le protocole HTTP

- Nous sommes dans l'architecture client-serveur:
 - Le client envoie une requête au serveur
 - Le serveur envoie une réponse au client



- Une technologie comme Ajax va nous permettre d'échanger des données entre le client et le serveur.

Rappels sur le protocole HTTP



- **Structuration d'une requête**

Une requête est composée d'une entête et d'un corps:

- L'entête contient des informations utiles au serveur pour son traitement.
- Le corps contient des informations à traiter.

- ***Format d'une requête***

Ligne de commande (Commande, URL, Version de protocole)

En-tête de requête

[Ligne vide]

Corps de requête

- ***Corps***

- peut contenir sous un format donné (facultatif)

Rappels sur le protocole HTTP



- **Structuration d'une réponse**

Une réponse, comme une requête est composée une entête et d'un corps:

- L'entête contient des informations utiles au client pour son traitement.
- Le corps contient des informations a donner au client

- ***Format de l'entête pour la réponse***

Ligne de statut (Version, Code-réponse, Texte-réponse)

En-tête de réponse

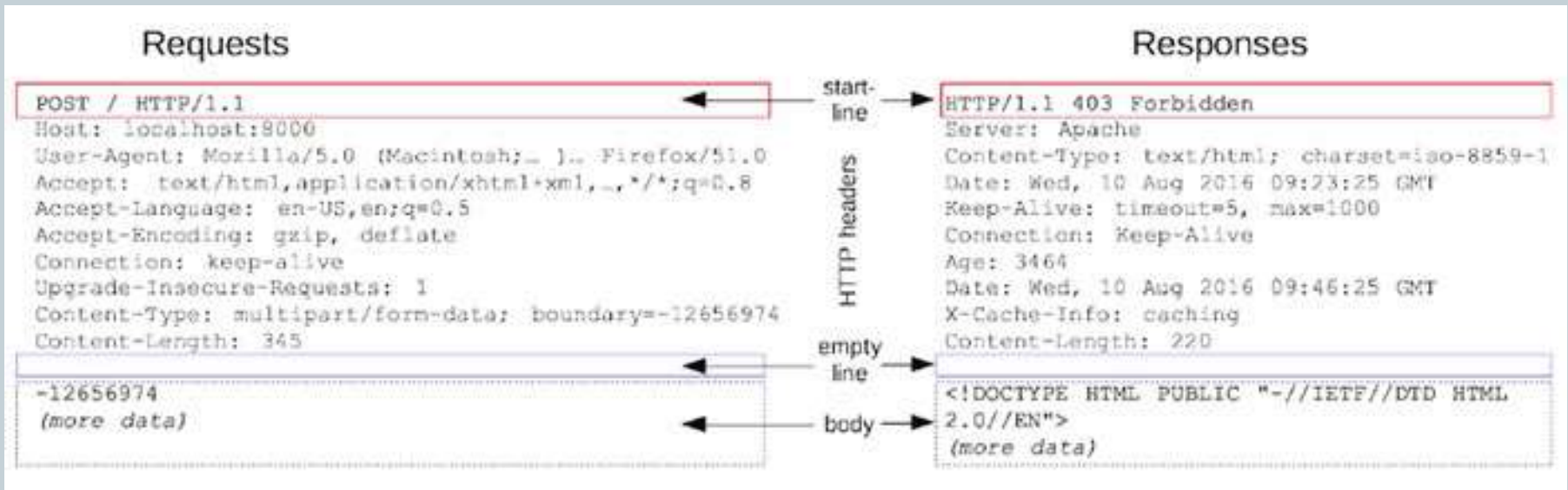
[Ligne vide]

Corps de réponse

- ***Corps***

- contient les données HTML de la page envoyée

Rappels sur le protocole HTTP



Pour analyser le contenu d'une requête ou d'une réponse, rendez vous sur la partie network des outils associées au différents navigateurs.

Rappels sur le protocole HTTP



- **Les méthodes HTTP**

GET

La méthode GET demande une représentation de la ressource spécifiée. Les requêtes GET doivent uniquement être utilisées afin de récupérer des données, donc demande les données en mode lecture et non écriture.

HEAD

Identique à une requête GET, mais nous demandons que l'en-tête, pas le corps.

POST

La méthode POST est utilisée pour envoyer une des informations vers le serveur

PUT

La méthode PUT remplace toutes les représentations actuelles de la ressource visée par le contenu de la requête – modification totale de la ressource .

PATCH

La méthode PATCH est utilisée pour appliquer des modifications partielles à une ressource.

DELETE

La méthode DELETE supprime la ressource indiquée.

Rappels sur le protocole HTTP



- **Les codes de statuts HTTP**

Ces codes permettent de connaître l'état de réalisation de la requête:

En résumé:

1xx: donne des informations

2xx: état de succès

3xx: redirection

4xx: erreur client

5xx: erreur serveur

REST: contraintes architecturales

- **Architecture client-serveur**
- **Sans état:** Une application sans état ne maintient pas de connexion ni ne stocke d'informations entre deux requêtes du même client. Un client fait une requête, l'API exécute l'action définie dans la requête et répond. Une fois que l'API a répondu, elle se déconnecte et ne conserve aucune information sur le client dans sa mémoire active. L'API traite chaque requête comme une première demande.
- **Avec mise en cache:** Une REST API doit normalement permettre la mise en cache des données fréquemment demandées. Pour réduire la bande passante, la latence et la charge du serveur, une API doit pouvoir identifier les ressources pouvant être mises en cache, déterminer qui peut les mettre en cache et décider pendant combien de temps elles peuvent rester dans le cache.
- **Interface uniforme:** Le client interagit avec le serveur selon une manière définie, indépendamment de l'appareil ou de l'application.
- **Identification des ressources :** L'API doit avoir un URI (identifiant de ressource uniforme) spécifique pour chaque ressource, tel que `/monitor/{monitorGuid}`
- **Auto-descriptif:** Comprend des métadonnées telles que Content-Type qui décrit comment interpréter la réponse.
- **Système en couches:** Une API peut avoir plusieurs couches, telles que des serveurs proxy ou des dispositifs de répartition de charge, et le serveur d'extrémité peut déployer des serveurs supplémentaires pour formuler une réponse. Le client ne sait pas quel serveur répond à la requête. Un système en couches rend une API plus évolutive.

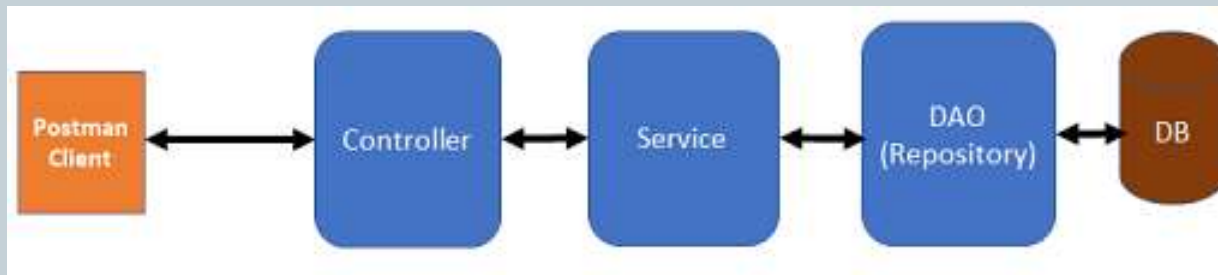
Structure par couches d'une application Spring



Une application Spring est constituée de plusieurs couches:

- Les **Repositories**: permettent les accès en lecture et écriture dans la base de données.
- Le **Services**: développent toutes les fonctionnalités métier de l'application.
- Les **Controllers** : gèrent le trafic HTTP en recevant une requête et en renvoyant une réponse

Ces différentes couches interagissent d'une façon précise, schématisées ci-dessous:



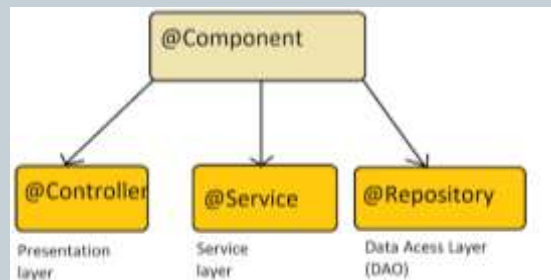
Remarque: *on créera des packages différents* pour chacune de ces couches, histoire de s'y retrouver!

Structure par couches d'une application Spring



Ces différentes couches sont constituées de classes remplissant les fonctions propres associées. Spring utilise les annotations pour distinguer les fonctionnalités des différentes couches. Ces classes sont en fait des **beans** dont le cycle de vie est régi par la mécanique de Spring. Les beans sont des objets Java créés au moment du démarrage de l'application.

- **Les controllers** – Annotation: **@Controller**
- **Les services** – Annotation : **@Service**
- **Les repositories** – Annotation: **@Repository**



Ces 3 annotations, sont des spécialisations de @Component et ne sont que des particularisations sémantiques de celle-ci...

Arborescence d'un projet Spring Boot



- >  src/main/java
- ✓  src/main/resources
 -  static
 -  templates
 -  application.properties
- >  src/test/java
- >  JRE System Library [JavaSE-17]
- >  Maven Dependencies
- >  src
- >  target
-  HELP.md
-  mvnw
-  mvnw.cmd
-  pom.xml

Architecture d'une application Spring Boot - Maven



Maven gère les dépendances de notre projet par rapport aux différentes librairies intégrées.

La construction (build) du projet sera gérée par Maven.

Maven peut générer le projet sous forme de jar (archive java) ou war (web archive)

Maven s'appuie sur un fichier en format xml le pom (Project Object Model) – les dépendances ainsi que les façons de les intégrer dans le pom pourront être obtenues sur :

<https://mvnrepository.com/>

Lors de la création d'un projet, Maven demande le groupe id (un identifiant de l'organisation qui a créé le projet, sous la forme d'un nom de domaine), artifactId (ce peut être le nom du projet, et ce sera le nom du jar), et la version du projet.

Architecture d'une application Spring Boot

–Maven: Structure du pom.xml

présentation du projet:

<project>

 <modelVersion>4.0.0</modelVersion>

 <groupId>com.mycompany.app</groupId>

 <artifactId>my-app</artifactId>

 <version>1</version>

fichiers à compiler: <build> <</sourceDirectory> </build>

Gestion des dépendances:

<dependencies> : gestion des dépendances

 <dependency>

 <groupId>com.fasterxml.jackson.datatype</groupId>

 <artifactId>jackson-datatype-jsr310</artifactId>

 </dependency>

 <dependency>

 <groupId>commons-io</groupId>

 <artifactId>commons-io</artifactId>

 <version>1.4</version>

 </dependency>

</dependencies>

</project>

Le fichier de configuration application.properties



Exemples: configuration du logger, accès à la base de donnée etc.

Configuration

– Spring Boot se configure via le fichier src/main/resources/**application.properties**”

Exemple de configuration

```
# datasource
spring.datasource.url = jdbc:h2:mem:testdb
spring.datasource.driver-class-name = org.h2.Driver
spring.datasource.username = sa
spring.datasource.password =
spring.jpa.generate-ddl=true
# mvc
spring.mvc.view.prefix = /pages/
spring.mvc.view.suffix = .html
```


Architecture d'une application Spring Boot



Point d'entrée dans le programme:

le projet Spring Boot développé s'exécute comme une application Java standard. Le point d'entrée du programme est la méthode `main()`.

```
@ComponentScan({« package.controller", "package.service", "package..listener"})
@EntityScan("package..entity")
@EnableJpaRepositories("dpakage.repository")
@SpringBootApplication
public class SgpWebapiApplication {
    public static void main(String[] args) {
        SpringApplication.run(SgpWebapiApplication.class, args);
    }
}
```

Un serveur HTTP Tomcat par défaut est alors exécuté et les web services développés transitent vers lui!

Architecture d'une application Spring Boot



Rappel sur les annotations Java

Définition

Les annotations en Java sont des marqueurs qui permettent d'ajouter des méta-données aux classes, aux méthodes, aux attributs, aux paramètres, aux variables, aux paquets ou aux annotations elles-mêmes.

Notation

@Nom_annotation () avec des paramètres éventuels pour faciliter certains traitements.

Positionnement

Une annotation peut se trouver au niveau d'un :

- D'une classe
- Attribut
- D'une méthode
- De l'argument d'une méthode

Architecture d'une application Spring Boot



Nous remarquons un certain nombre d'annotations dans cette classe:

@ComponentScan("package.controller","package.service","package.listener"): parcourt les beans contenus dans les différents packages pour pouvoir les réutiliser.

@EntityScan("package.entity"): idem mais spécialisé pour les entités en base de données.

@EnableJpaRepositories("package.repository"): permettre l'usage des répositories JPA.

@SpringBootApplication: *se charge du scan des packages et des configuration de base.*

Architecture d'une application Spring Boot



Nous développerons sous le mode API, et il est important d'éviter certains problèmes de sécurité, en particulier ceux relatifs aux origines croisées de sollicitation des web services sur l'api: **Cross-Origin Resource Sharing (CORS)**

Afin de résoudre pour un premier niveau ce type de problème, nous permettrons les origines multiples:

@Bean

```
public WebMvcConfigurer corsConfigurer() {  
    return new WebMvcConfigurer() {
```

@Override

```
        public void addCorsMappings(CorsRegistry registry) {  
            registry.addMapping("/api/**")  
                .allowedHeaders("Origin", "Content-Type", "Accept")  
                .allowedOrigins("**")  
                .allowedMethods("**");  
        }  
    };
```

```
}
```

Les contrôleurs- notions de base



- Un contrôleur est une classe Java ***permettant de de réceptionner une requête HTTP et d'envoyer une réponse***
- Un contrôleur est annoté au niveau de la classe par l'annotation ***@Controller***
- C'est aussi au niveau de cette classe contrôleur que l'on peut compléter l'url de base des web services

Notion de contrôleur- exemple déclaration d'un controller



Il est possible de factoriser au niveau de la classe le complément de l'url de base du serveur afin de spécifier davantage le traitement attendu des webservices.

Pour cela, on utilise l'annotation

@RequestMapping(« ici, le complément d'url »)

Exemple de complément d'url:

@RequestMapping ("/api")

Notion de contrôleur- exemple déclaration d'un controller



@Controller

@RequestMapping("/api/employees")

```
class EmployeeController {
```

```
...
```

```
}
```

Donc toutes les autres routes qui seront définies au niveau du « controller » seront précédées de /api/employees. Ce complément étant précédé par l'adresse de base du serveur – par exemple :

http://localhost:8080/api/employee

Notion de contrôleur- exemple déclaration d'un controller



Transfert des données entre le client et le serveur

Maintenant, selon les web services relatifs au projet développé, la classe contiendra des méthodes qui gèrent spécifiquement tel ou tel requête.

D'une façon générale, nous travaillerons avec ***le format JSON***. Une information transmise sur le réseau sera automatiquement « ***Stringifiée*** » par Jackson (parser JSON), et une chaîne de caractères JSON reçue par notre api sera ***parsée*** via la même librairie.

Notion de contrôleur- exemple

déclaration d'un controller



Nous solliciterons ces méthodes via ***un complément d'url spécifique précisé par les annotations suivantes:***

- ***@GetMapping – méthode HTTP GET, récupération de données***
- ***@PostMapping – méthode HTTP Post, enregistrement de données***
- ***@PutMapping – méthode HTTP Put, modification des données***
- ***@DeleteMapping – méthode HTTP Delete, suppression des données***
- ***...***

Notion de contrôleur - exemple d'implémentation d'un controller

Nous pourrions nous retrouver avec cela:

@RestController

@RequestMapping("/api/employees")

```
class EmployeeController {
```

```
// méthode permettant de retrouver tous les employés // de la base de données
```

```
@GetMapping("/employees")
```

```
List<Employee> all() {
```

```
    return employeeService.getEmployees(); // le service est appelé dans le  
    contrôleur }
```

```
@PostMapping("/employees") Employee newEmployee(@RequestBody
```

```
Employee newEmployee) { return repository.save(newEmployee); }}}
```

Dans le navigateur: on entrera l'url : <http://localhost:8080/api/employee/employees>

Ici, comme les deux méthodes HTTP sont différentes, bien que l'on ait deux url identiques, il n'y a pas d'ambiguïté dans le mapping des urls

Notion de contrôleur - exemple d'implémentation d'un controller



@RequestBody : nous permet *au niveau des paramètres* d'une méthode de contrôleur de récupérer le corps de la requête HTTP. Il s'agit généralement des données transmises lors de l'envoi d'un formulaire en méthode HTTP "POST". Il en est de même lors d'un PUT. Les données sont formatées généralement en JSON, XML, etc.

Attention: côté front bien préciser le **contentType** via AJAX!

Notion de contrôleur - exemple d'implémentation d'un controller

@PathVariable : nous permet de récupérer, toujours **au niveau des paramètres** d'une méthode, la valeur de la variable de chemin contenue dans l'url du web service. Ces url sont définies dans le cadre de Spring de la façon suivante: par /api/user/{id}

Par exemple:

```
@GetMapping("/api/user/{id}")  
public User getById(@PathVariable("id") int id) {  
    return this.userService.findById(id);  
}
```

Notion d'ORM



Un **ORM** (**O**bject **R**elational **M**apper) est un outil qui permet d'établir une correspondance entre une base de donnée relationnelle et un langage orienté objet.

Typiquement, à une classe pourra correspondre une table, et les différents attributs de la table correspondront aux différents attributs de la classe.

Dans la pratique, il nous sera possible grâce à un ORM d'opérer des manipulations en lecture et écriture sur la base de données via des méthodes Java.

Exemple: `repository.save(e)` sauvegardera l'employé représenté par l'objet `e`.

Génération des tables via JPA



La puissance de l'ORM JPA nous permet ***via le langage Java*** de donner les instructions pour la création des tables dans la base de données.

Cette création est gérée par un ensemble d'annotations spécifiques.

En partant de l'idée de la correspondance Objet – Relationnel présentée ci-dessus, JPA propose un certain nombre d'annotations permettant de réaliser cette correspondance en partant du modèle. Nous pouvons retenir:

- @ Entity
- @Table
- @Column

- Viennent s'adjoindre à ces annotations, d'autres plus spécifiques permettant la représentation des relations entre les entités présentes dans la base de données (one to many, one to one, etc).

Génération des tables via JPA



@Entity // préparer cet objet au stockage dans un magasin de données basé sur JPA.

@Table(name="employee") // *permet de préciser que la classe générera une table – le nom de cette table est précisé entre parenthèses*

```
public class Employee{
```

@Id // *déclaration d'un id*

@GeneratedValue(strategy = GenerationType.AUTO) // id auto incrémenté

```
private int id;
```

@Column(name="matricule") // création d'une colonne « matricule » en bdd

```
private String matricule;
```

@Column(name="nom") // création d'une colonne « nom » en bdd

```
private String nom;
```

@Column(name="prenom") // création d'une colonne « prénom » en bdd

```
private String prenom;
```

```
...}
```

Il est à noter que nous ne sommes pas contraints de rajouter les names au niveau des colonnes et tables si l'on souhaite garder les mêmes noms que ceux spécifiés en Java

Les repositories JPA



Il existe un très grand nombre d'ORM. Nous illustrerons ici l'usage de JPA (**J**ava **P**ersistence **A**PI) via les Repositories.

La notion de repository nous permettra de faciliter les manipulations en base de données – grâce à un grand nombre d'utilitaires fondés sur la notion d'ORM et d'opérations pré-implémentées.

Création d'un Repository



Dans le cadre de JPA, un Repository est une interface qui va hériter de « JpaRepository ».

Etant une interface, aucune opération de cette interface n'est implémentée! Le travail de traduction de la déclaration de l'opération se fait « en arrière plan » sans que nous ayons à faire autre chose qu'une déclaration...

Création d'un Repository



Déclaration d'un Repository:

@Repository

```
public interface EmployeeRepository extends JpaRepository<Employee,  
    Integer> {  
    ...  
}
```

Techniquement: bien préciser l'annotation **@Repository** et étendre l'interface avec **JpaRepository**<**Employee**, **Integer**>

Le Repository est spécifié ainsi : Repository<T, ID>. T étant le type de l'objet géré par le Repository et ID étant le type de la clé de l'objet.

Création d'un Repository



@Repository

```
public interface EmployeeRepository extends  
    JpaRepository<Employee, Integer> {  
    List<Employee> findById(int id);  
    List<Employee> findByName(String nom);  
}
```

Il n'est pas besoin de préciser davantage!

Effectivement JPA va analyser le nom de l'opération en tant que chaîne de caractères. Dans l'exemple donné ici:

- find effectuera l'équivalent d'un « select »,
- le mot clé « By » va préciser sur quel attribut va s'effectuer la recherche (joue le rôle de filtre)
- le paramètre de l'opération va donner la valeur à rechercher...

Création d'un Repository



Nous aurions pu aussi définir dans l'interface les opérations suivantes:

```
Employee getByLogin(String login); // recherche un employé par son login
```

```
long countByEmail(String email);
```

```
List< Employee > findByNameAndEmail(String name, String email); //  
rechercher par nom et par email
```

```
List< Employee > findByNameOrEmail(String name, String email);  
// recherche par nom ou par email
```

Il y a beaucoup de possibilités. Il faut se référer à la documentation pour s'en rendre compte.

De plus une opération de l'interface peut être annotée par **@Query** qui permet de spécifier des requêtes plus spécifiques que celles prévues par défaut dans le repository

Création d'un service



Comme dit précédemment, le service va effectuer les opérations métier exigées par notre projet

Création d'un service



```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository employeeRepository;

    public Employee getEmployee(final Long id) {
        return employeeRepository.findById(id);
    }
    public Iterable<Employee> getEmployees() {
        return employeeRepository.findAll();
    }
    public void deleteEmployee(final Long id) {
        employeeRepository.deleteById(id);
    }
    public Employee saveEmployee(Employee employee) {
        Employee savedEmployee = employeeRepository.save(employee);
        return savedEmployee;
    }
}
```

Les beans Spring



Nous voyons dans ce code, une nouvelle annotation **@Autowired**. Elle permet de récupérer automatiquement un bean de Spring dans une classe donnée, ici une classe de type Service. Ici, nous voulons une instance du Repository qui est déclarer dans la classe de service comme une dépendance. Avec cette annotation nous n'avons pas besoin de getters ou de setters.

Si vous n'avez pas déclaré le Repository comme Bean (via une annotation de type **@Repository**), le conteneur Spring ne peut pas injecter la dépendance dans notre Service et on a donc une exception.

Il en est de même pour les différentes autres annotations que nous avons vues: dans le contrôleur vous trouverez un bean relatif au Service, dans celui de du Service un bean relatif au Repository

Les beans Spring



Un Bean Spring est un objet qui est instancié, assemblé et géré par Spring IoC Container. IoC (Inversion of control), est un processus qui définit les dépendances d'un objet sans avoir à les créer. C'est lors de la création des objets, que Spring va injecter les Beans entre eux afin d'avoir toutes leurs dépendances.

Un bean Spring a 3 grandes phases:

- **Initialisation:** le bean est créé par le conteneur Spring , ses dépendances sont résolues
- **Usage:** le bean est utilisable dans l'application
- **Destruction:** les beans sont envoyés au garbage collector

Sollicitation d'une l'API via Ajax



AJAX- *Asynchronous JavaScript And XML*

- **Buts d'Ajax**

Grâce à Ajax, nous pourrons effectuer une requête via notre code JavaScript et traiter la réponse du serveur :

- Recevoir des données d'un serveur
- Envoyer des données à un serveur

Il est tenu en compte le temps que prend les communications entre le client et le serveur pour séquencer les actions.

- **L'objet d'XMLHttpRequest**

Définition:

XMLHttpRequest définit une API qui prend en charge le transfert de données entre un client et un serveur. Plus précisément : Une requête http est envoyée au serveur. Un objet appelé d'XMLHttpRequest va recueillir les données envoyées par le serveur. Les navigateurs modernes prennent tous en charge cet objet...

Sollicitation de l'API via Ajax



L'api XMLHttpRequest

- La méthode ***open(method, url, mode, login, password)***

But : Elle permet d'établir une connexion avec le serveur

La méthode open prend en paramètres :

- La méthode HTTP utilisée : GET, ou POST, etc
- L'url du service/ du fichier qui contient les données
- Le mode : c'est un booléen, soit asynchrone – true ou synchrone – false – ce paramètre est optionnel, par défaut le mode est asynchrone.
- Login : ce paramètre est optionnel
- Password : ce paramètre est optionnel

Donc d'une façon générale, on aura :

Ou au plus simple : ***open(method, url)***

Sollicitation de l'API via Ajax



- **La méthode send**

But : Elle envoie la requête au serveur

La méthode send prend en paramètres les données à envoyer : si la méthode HTTP est GET alors, on met null.

Sollicitation de l'API via Ajax



Récupération de la réponse

Deux attributs de l'objet XMLHttpRequest permettent de récupérer les données qui sont dans la réponse en provenance du serveur

- l'attribut responseXml

La réponse est sous la forme d'une chaîne de XML

- l'attribut.responseText

La réponse est sous la forme d'une chaîne de caractères

Statut de la requête HTTP

Un attribut de l'objet XMLHttpRequest : status, permet de connaître le statut de la réponse envoyée par le serveur.

Sollicitation de l'API via Ajax



- **Etat du traitement de la requête**

- Un attribut de l'objet XMLHttpRequest permet de connaître la progression de la réalisation de la requête : **readyState**

état	description
0	<i>non initialisée</i>
1	<i>connexion établie</i>
2	<i>requête reçue</i>
3	<i>réponse en cours</i>
4	<i>terminée</i>

- Par exemple, quand le statut http est 2xx et quand la valeur de **readyState** est 4, alors la réponse est prête à être exploitée...
- **Changement de l'état de traitement de la requête**
- La propriété **onreadystatechange** spécifie une fonction à exécuter chaque fois que le statut de l'objet XMLHttpRequest change.

Sollicitation de l'API via Ajax



- **Ajout d'informations dans le header**
- On utilise la méthode ***setRequestHeader*** pour ajouter des entêtes aux requêtes http.
- Attention: `setRequestHeader()` doit être appelé après `open()` et avant `send()`.
- En particulier, on précise le Content-Type pour un POST

Sollicitation de l'API via Ajax



- **Obtenir via chrome les informations venant du serveur**
- Par exemple, a la suite de la création d'un utilisateur, nous allons dans la console chrome et : ***network -> users->response*** et nous voyons la réponse envoyée par le serveur.
- C'est une chaîne de caractères JSON, par exemple :

```
{  
  "name": "charles hattant",  
  "job": "voyant",  
  "id": "187",  
  "createdAt": "2019-01-16T13:27:11.125Z"  
}
```

Sollicitation de l'API via Ajax



- **Obtenir via chrome les informations venant du serveur**
- Par exemple, a la suite de la création d'un utilisateur, nous allons dans la console chrome et : ***network -> users->response*** et nous voyons la réponse envoyée par le serveur.
- C'est une chaîne de caractères JSON, par exemple :

```
{  
  "name":"charles hattant",  
  "job":"voyant",  
  "id":"187",  
  "createdAt":"2019-01-16T13:27:11.125Z"  
}
```


Sollicitation de l'API via Ajax



Effectuer de multiples requêtes en boucle

- Nous sommes confrontés à un problème d'asynchronisme car **chaque requête demande du temps pour être effectuée**
- Il faut éviter que la boucle ne continue sans attendre que la requête en cours soit finie.
- On utilise pour cela dans la boucle une fonction d'exécution immédiate. Son corps contiendra la requête.

Exemple :

```
var response = JSON.parse(req.responseText);  
//le response.results est un tableau  
for (var k = 0; k < response.results.length; k++) {  
  (function(y) {  
    Requêtes en AJAX – comme vue précédemment  
  })(k);  
}
```

Sollicitation de l'API via Ajax



Voici un exemple qui utilise la méthode GET:

```
function loadPage() {  
  // création de l'objet xml http request:  
  var xhttp = new XMLHttpRequest();  
  xhttp.onreadystatechange = function() {  
    if (this.readyState == 4 && this.status == 200) {  
      document.getElementById("demo").innerHTML =  
      this.responseText;  
    }  
  };  
  xhttp.open("GET", "url_service", true);  
  xhttp.send();  
}
```